

CSE 214 ADVANCED APPLICATION DEVELOPMENT

TERM PROJECT:

FULL STACK E-COMMERCE WEB APPLICATION DEV

fibiyo



Prepared by: Burak Yalçın (Student ID: 20220808069)



Yahya Efe Kuruçay (Student ID:20220808005)



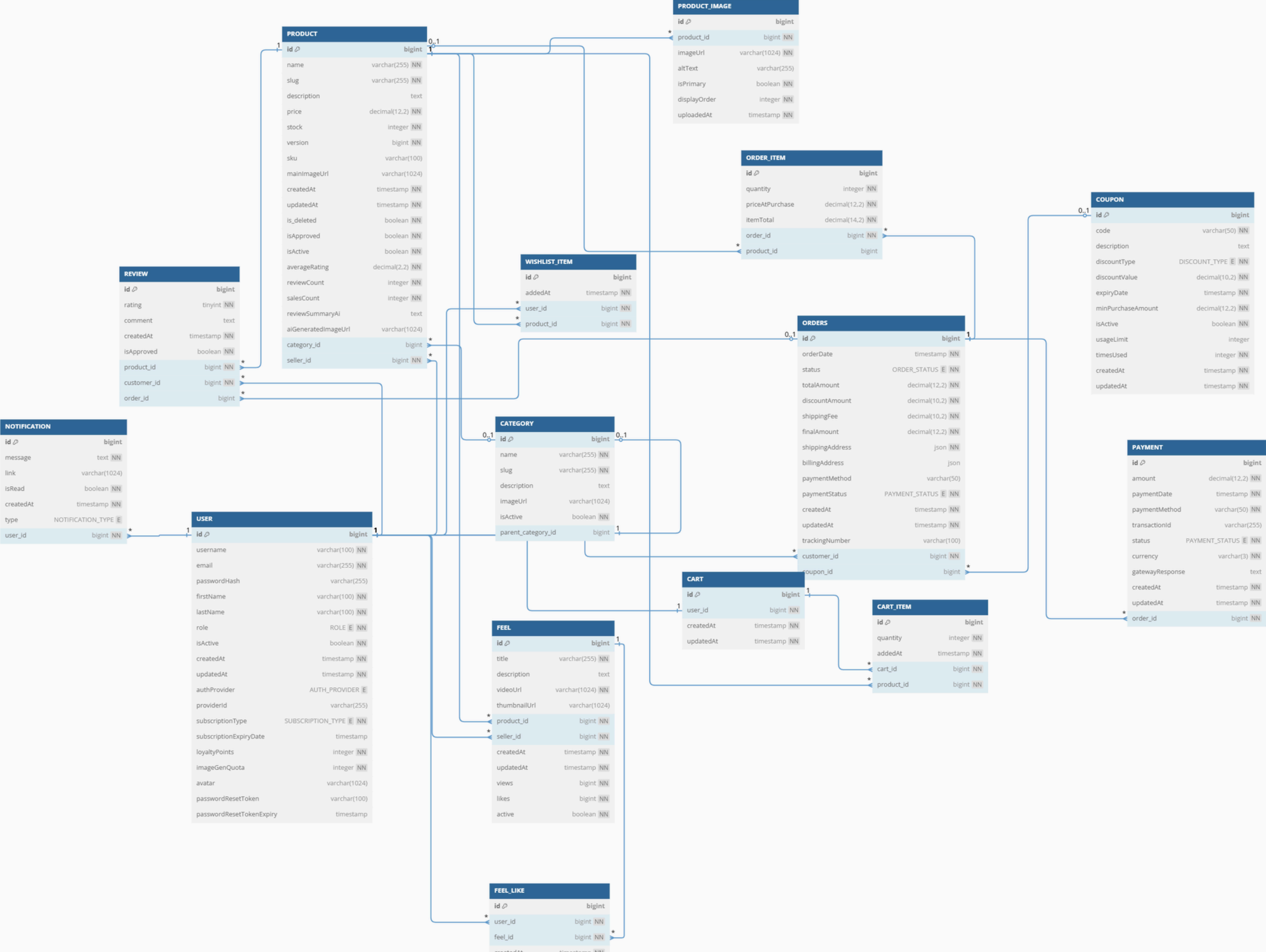
:

[github.com//agrotopya/ecommerce_with_angular_springboot_mysql/ecommerce_with_angular_springboot_mysql](https://github.com/agrotopya/ecommerce_with_angular_springboot_mysql/ecommerce_with_angular_springboot_mysql)

Introduction to Our Application

The Fibiyo E-Commerce Platform is a full-stack application architected with Angular for the frontend, Spring Boot for the backend, and MySQL as the relational database. The backend implements a RESTful API adhering to a layered architecture (Application, Domain, Infrastructure) and utilizes Spring Security with JWT for authentication and role-based authorization (Customer, Seller, Admin). Core functionalities include user registration and profile management, product cataloging with category hierarchy, AI-assisted image generation (OpenAI DALL-E/ChatGPT integration), "Feels" short video management, shopping cart operations, a secure checkout process leveraging Stripe for payment processing, and comprehensive order management. The system features distinct capabilities for Admins (user/product/order/coupon/review/feel management), Sellers (product/order/feel management, AI image tools, subscription benefits), and Customers (browsing, reviews, cart, orders, wishlist). The frontend is built with standalone components, signals for state management, and lazy-loaded feature modules, interacting with the backend via well-defined API endpoints. Data persistence is managed by Spring Data JPA, and MapStruct is utilized for DTO-entity mapping. The overall design prioritizes scalability, maintainability, and a modular approach to development.

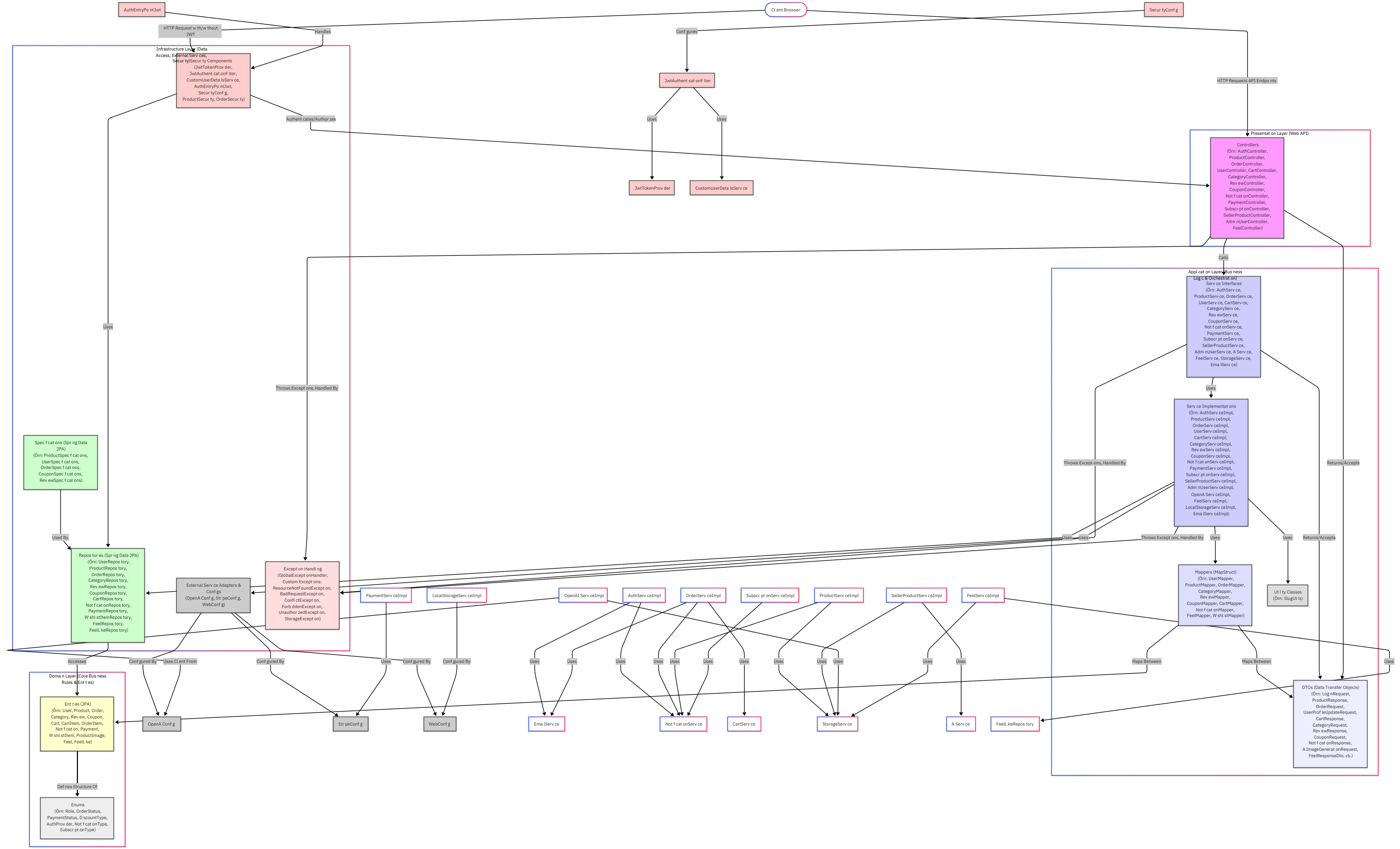
DATABASE ER DIAGRAM



Explaining of the ER Diagram

This Entity-Relationship (ER) diagram meticulously outlines the database schema for the e-commerce backend. At its core, the USER table manages customer, seller, and admin accounts, linking to their respective PRODUCTS (for sellers) and ORDER_TABLEs (for customers). Product information is detailed in the PRODUCT table, which maintains a hierarchical relationship with CATEGORY for organization and connects to PRODUCT_IMAGE for multiple visuals. The purchasing BLoC revolves around the ORDER_TABLE, which aggregates ORDER_ITEMS (detailing each product purchased at a specific price), and is associated with PAYMENT records and potentially COUPONS for discounts. User engagement features like REVIEWS for products, a WISHLIST_ITEM system, and NOTIFICATIONS are directly linked to users and products. A CART and CART_ITEM system facilitates the shopping BLoC before an order is placed. Furthermore, a content-driven feature is supported by FEEL (short video content) and FEEL_LIKE tables, allowing sellers to showcase products and users to interact. Relationships are primarily one-to-many (e.g., a user can have multiple orders) or one-to-one (e.g., a user has one cart), with appropriate foreign keys, unique constraints, and data types defined to ensure data integrity and facilitate efficient querying. Key attributes like timestamps, soft delete flags (for PRODUCT), and versioning (for PRODUCT) enhance data management and auditability.

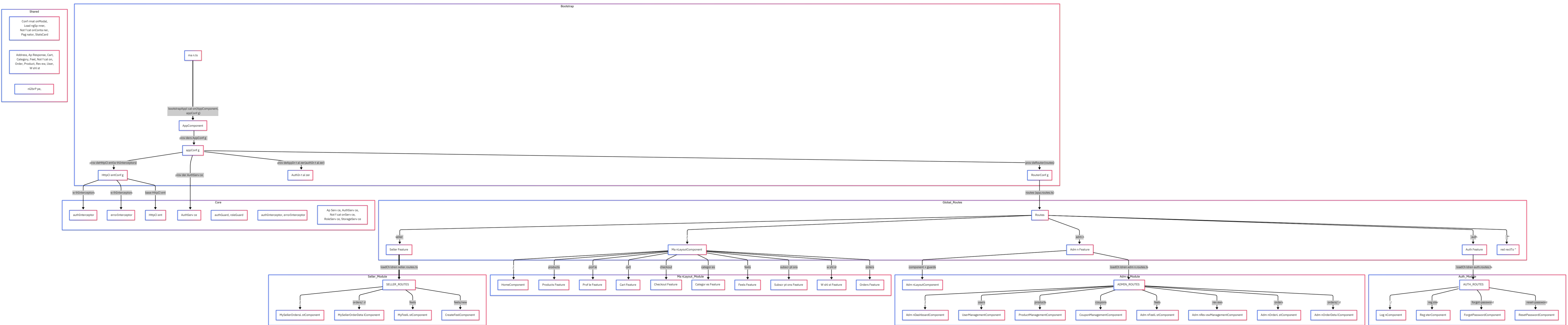
BACKEND DIAGRAM



Explaining of the Backend Architecture Diagram

The backend architecture for the Fibiyo E-commerce Marketplace, as illustrated in the diagram, employs a layered design to ensure modularity and maintainability. Client requests are initially handled by the Presentation Layer (Controllers), which delegates business operations to the Application Layer (Services and ServiceImpls). This layer orchestrates data transformations between DTOs (for API communication) and Entities (domain objects) using Mappers (MapStruct), and interacts with the Domain Layer which defines core business Entities and Enums. The Infrastructure Layer underpins these operations, providing data persistence through Repositories and Specifications, managing security via SecurityComponents (including Spring Security with JWT for authentication/authorization), facilitating integration with external services like Stripe and OpenAI through External Adapters & Configs, and centralizing error management with a GlobalExceptionHandler. This structured approach allows for clear separation of concerns, efficient data flow, and robust integration of essential e-commerce functionalities.

FRONTEND DIAGRAM



Explaining of the Frontend Architecture Diagram

The application is initialized in `main.ts` using `bootstrapApplication`, where `appConfig` supplies the router, HTTP client (with `authInterceptor` and `errorInterceptor`), and an authentication initializer. Global routing points to a `MainLayoutComponent` root that lazy-loads feature modules for Home, Products, Profile, Cart, Checkout, Categories, Feels, Subscriptions, Wishlist, and Orders, alongside standalone Auth, Admin, and Seller modules. The Core layer provides key services (`ApiService`, `AuthService`, `NotificationService`, etc.), route guards (`authGuard`, `roleGuard`), and HTTP interceptors to enforce security and error handling. `SharedModule` gathers reusable components (confirmation modal, loading spinner, paginator, stats card), data models (`Product`, `Category`, `CartItem`, `Order`, etc.), and pipes (`nl2br`, `time-ago`). Each feature module encapsulates its own components and routes, leveraging Angular's standalone components, dependency injection, and lazy loading to create a modular, scalable, and maintainable frontend architecture.

Final Remarks

The Fibiyo E-Commerce Platform stands as a testament to our dedication in crafting a secure, scalable, and deeply user-centric digital marketplace. By integrating cutting-edge technologies like Angular for a responsive frontend, Spring Boot for a robust backend, and MySQL for persistent data storage, we have engineered a rich e-commerce ecosystem. This system not only supports essential functionalities such as comprehensive user and product management, an intuitive shopping cart, and secure Stripe-powered checkout, but also introduces innovative features tailored to distinct Customer, Seller, and Admin roles.

Key differentiators include the "Feels" feature, empowering sellers with dynamic short-video product showcases to significantly enhance engagement, and AI-driven tools like seller-assisted image generation. Our architectural choices, emphasizing a layered design and RESTful API principles, ensure maintainability and future growth. Looking ahead, we are excited to further leverage AI capabilities by integrating features such as automated review summarization and optimization, aiming to continuously refine the user experience and provide deeper insights. This project successfully meets contemporary e-commerce demands while establishing a strong foundation for ongoing innovation and the incorporation of advanced, value-driven functionalities.

